浙江大学

# 本科实验报告

| | |
|---|---|
| 课程名称： | 实验 lab1：RISC-V 浮点加操作指令及硬件扩展设计 |
| 姓　　名： | |
| 学　　院： | 信息与电子工程学院 |
| 系： | |
| 专　　业： | 信息工程 |
| 学　　号： | |
| 指导教师： | 赵武锋、钟睿 |

2025 年 12 月 9 日

专业：信息工程
姓名：
学号：
日期：2025.12.9
地点：玉泉校区

# 浙江大学实验报告

课程名称：计算机设计与组成
指导老师：赵武锋、钟睿
成绩：
实验名称：实验 lab1：RISC-V 浮点加操作指令及硬件扩展设计

## 一、实验目的与任务

1．完成 EL2 的指令集及硬件集成，使之支持单精度的基于 IEEE 754 标准的浮点加操作。

2．环境搭建的学习，环境搭建和 project 本身同等重要

## 二、实验过程

### （一）环境搭建

#### 1．安装 WSL2 系统：

（1）Wsl2 是 Windows Subsystem for Linux 2 的简称，是 Windows10/11 支持的轻量级 Linux 虚拟机，非常适合中小型体量的 Linux 工程搭建和运行。

（2）以管理员身份打开 PowerShell（没有安装 PowerShell 的同学也可以用自带终端），终端输入 wsl --install，终端输入 exit 退出至 windows，在 windows 终端运行 wsl --list，windows 终端输入 wsl 或 bash 即可重新进入 Ubuntu，终端中可通过运行 sudo su 并输入你的密码，以提高你的权限。

#### 2．挂载 VeeRwolf 工程目录并下载 project 需要的额外工具

（1）安装 project 所需的额外工具：

①Catapult-studio：此为 RISC-V 编译平台，非必须，可选择性安装

②Verilator，此为仿真器

③Gtkwave，此为波形显示器

④检查环境配置是否成功

（2）运行 verilator --version，如果没有报错，显示了版本号，则说明 verilator 安装成功；运行 gtkwave，如果没有报错，弹出新窗口，说明 gtkwave 安装成功

### （二）实验说明

## 1．译码级及信号修改

（1）Create a new structure fp_pkt_t，包括 fp_add, fp_mul, fp_div

```
6   //You may create the fp_pkt_t structure here
7   typedef struct packed {
8                         logic fp_add;
9                         logic fp_mul;
10                        logic fp_div;
11                         } fp_pkt_t;
```

（2）Create three newbits,called fpadd,fp mul,and fp div,that are part of the structure type el2_dec_pkt_t.

```
210 typedef struct packed {
211                       logic fp_add;
212                       logic fp_mul;
213                       logic fp_div;
214                       logic clz;
215                       logic ctz;
216                       logic cpop;
```

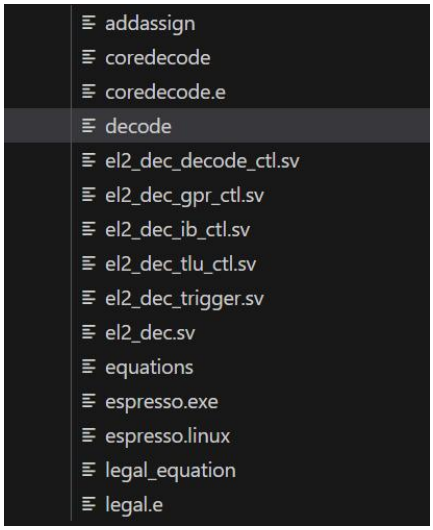
（3）Insection.definition， create a new line for each of the new instructions according to their format. Shown above

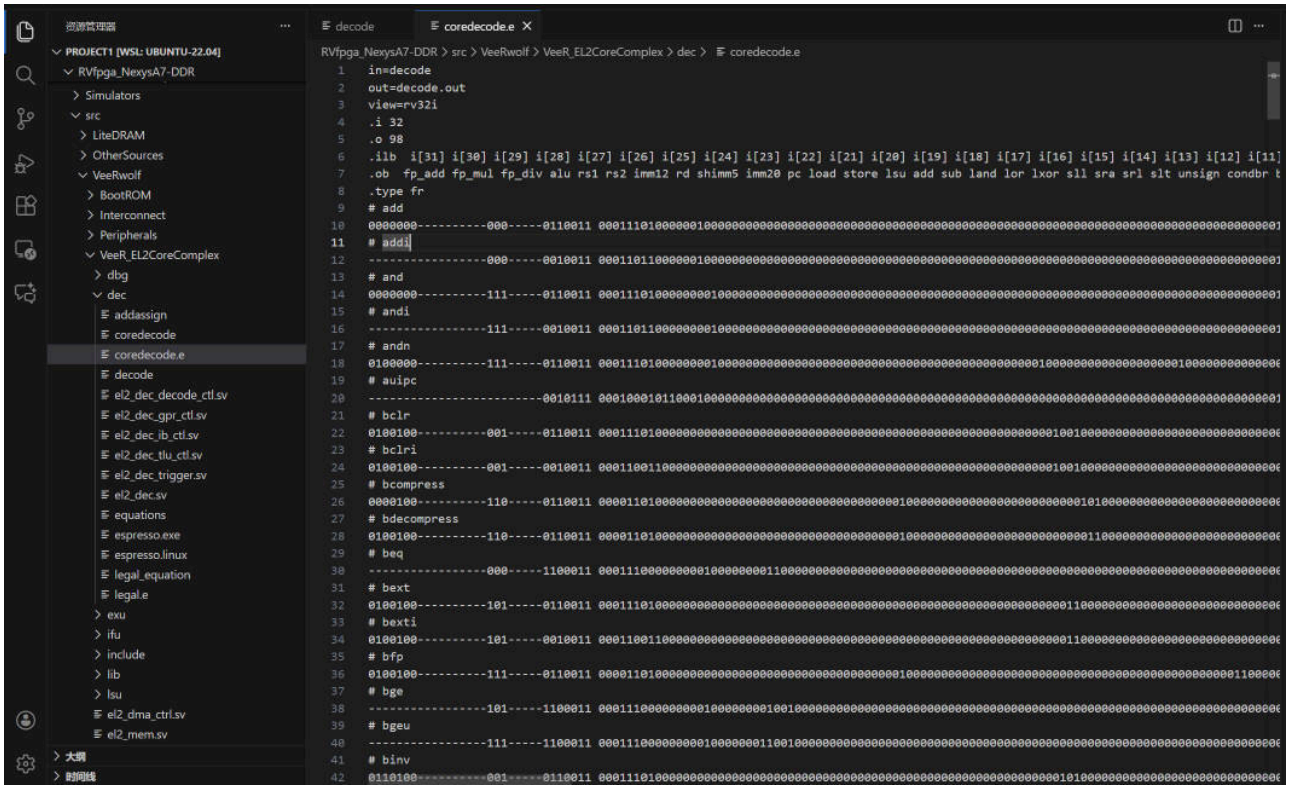
```
1
2   .definition
3
4   fadd      =  [0000000.................1010011]
5   fmul      =  [0001000.................1010011]
6   fdiv      =  [0001100.................1010011]
7
```

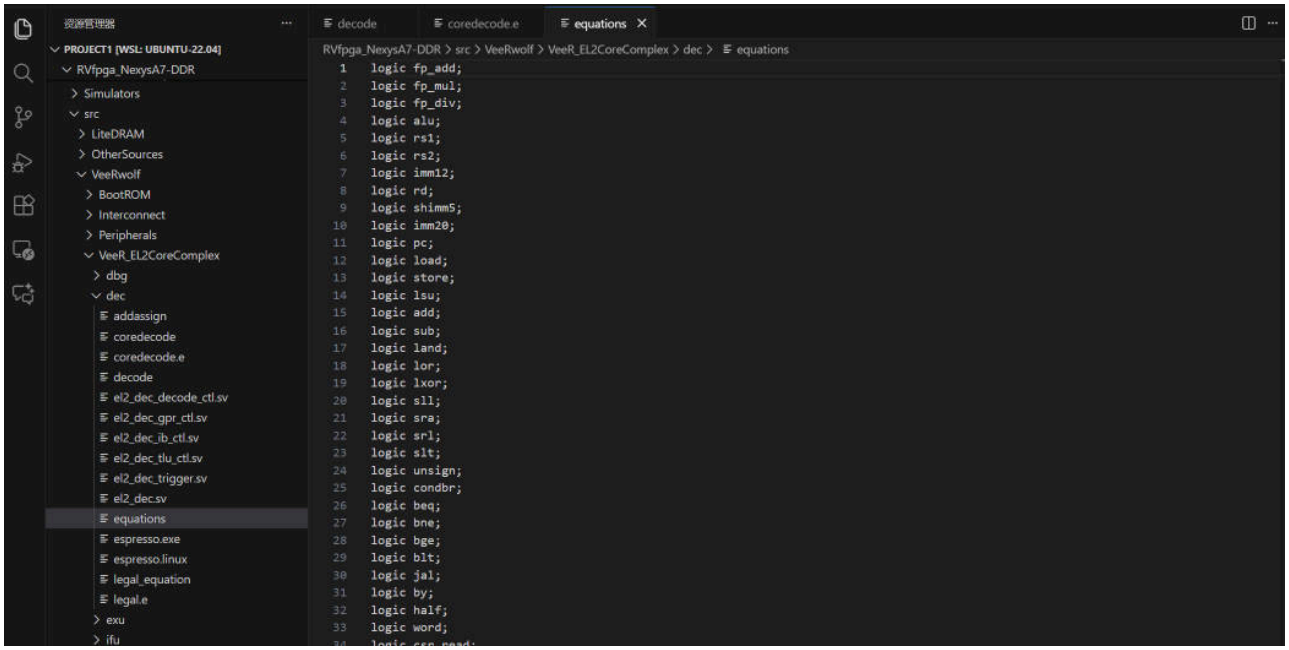
（4）Insection.output,create a new bit for each instruction

```
438 .decode
439 rv32i[fadd]      =  { div rs1 rs2  rd          presync postsync  fp_add  }
440 rv32i[fmul]      =  { div rs1 rs2  rd          presync postsync  fp_mul  }
441 rv32i[fdiv]      =  { div rs1 rs2  rd          presync postsync  fp_div  }
```

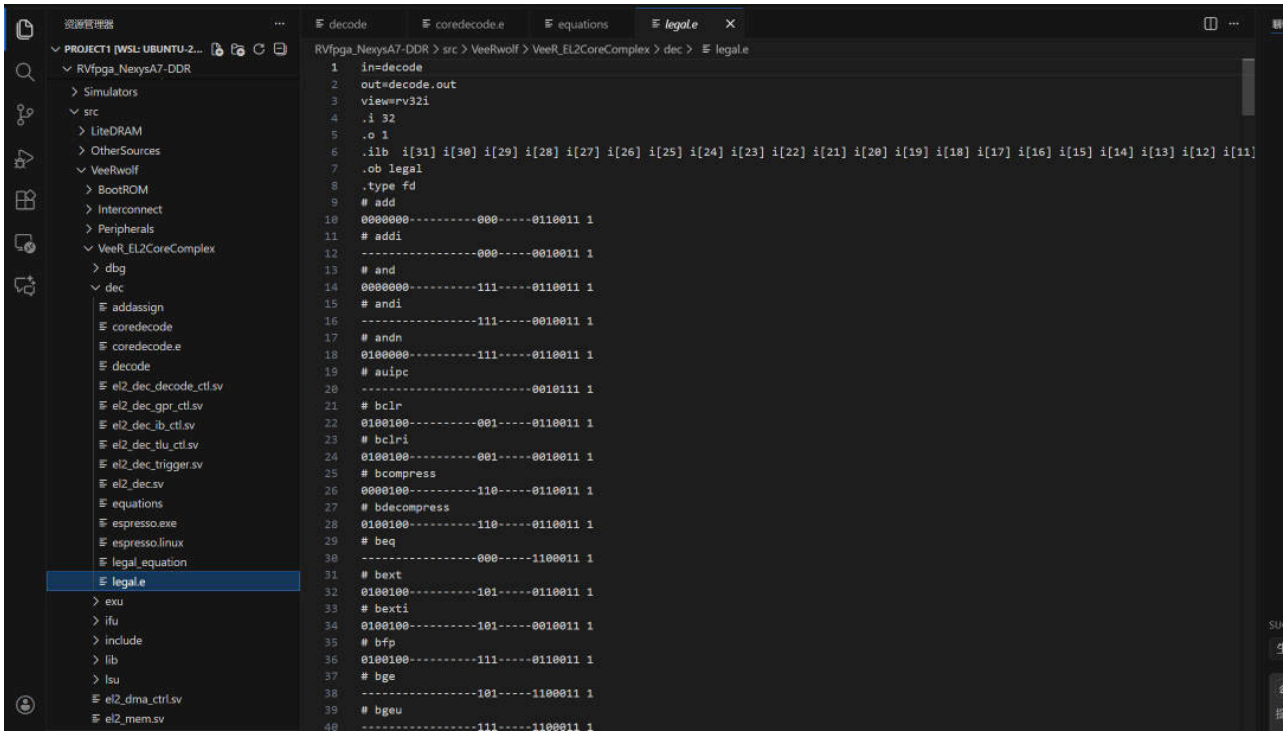
（5）generate the general equations, which, after the modification of the decodefile, will include the instructions supported by VeeR EL2 plus the three floating point instruction.

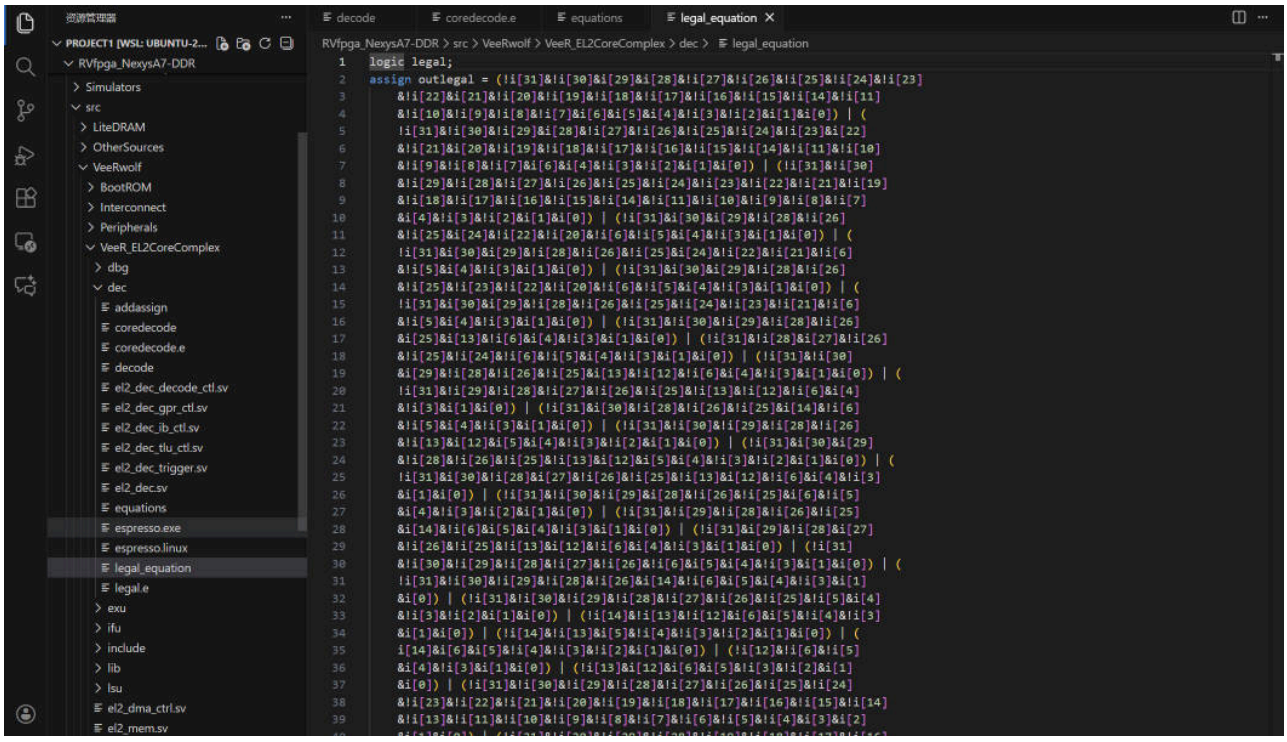




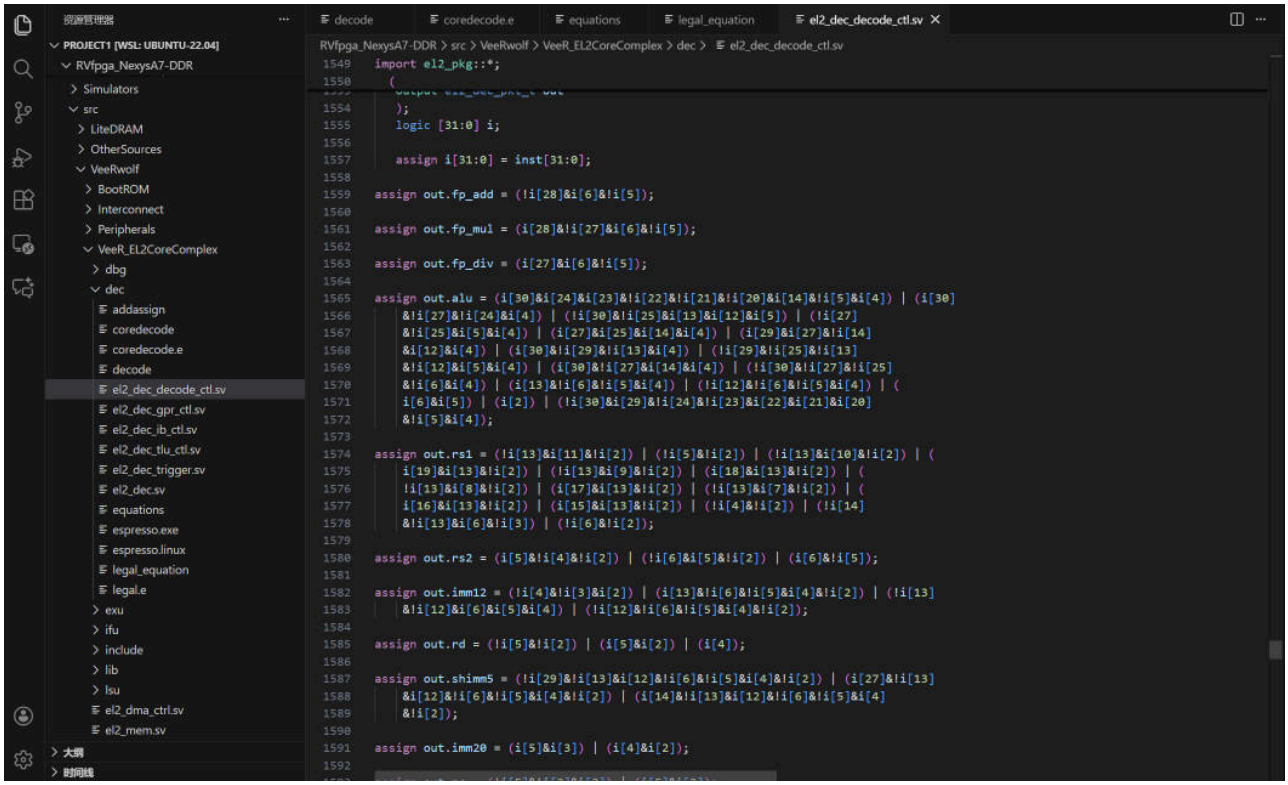




（6）Generate the legal equations， These two commands will generate files legal.e and legal_equations




（7）替代原有的 equations 和 legal_equations，Substitute the old equations with the new ones generated in the previous step， Specifically,you must substitute the existing equations in lines 1554-1862 of file el2_dec_decode_ctl.sv




（8）Assign a value to the new floating point control bits in signal fp_p,using signal io_dp

```
//FP
assign fp_p.fp_add    =  i0_dp.fp_add;
assign fp_p.fp_mul    =  i0_dp.fp_mul;
assign fp_p.fp_div    =  i0_dp.fp_div;

assign mul_p.valid    =  mul_decode_d;

assign mul_p.rs1_sign     =  i0_dp.rs1_sign;
assign mul_p.rs2_sign     =  i0_dp.rs2_sign;
assign mul_p.low          =  i0_dp.low;
assign mul_p.bcompress    =  i0_dp.bcompress;
```

（9）在 el2_dec_decode_ctl.sv 的开头输出信号中，加入 fp_p

```
output el2_div_pkt_t    div_p,                  // divide packet
output fp_pkt_t         fp_p,
output logic [4:0]       div_waddr_wb,           // DIV write address to GPR
output logic             dec_div_cancel,         // cancel the divide operation

output logic        dec_lsu_valid_raw_d,
output logic [11:0] dec_lsu_offset_d,
```

（10）el2_dec.sv 的输出信号中，加入 fp_p

```
output el2_lsu_pkt_t    lsu_p,                  // lsu packet
output logic             dec_qual_lsu_d,         // LSU instruction at D.  Use to quiet LSU operands
output el2_mul_pkt_t    mul_p,                  // mul packet
output el2_div_pkt_t    div_p,                  // div packet
output fp_pkt_t         fp_p,
output logic             dec_div_cancel,         // cancel divide operation

output logic [11:0] dec_lsu_offset_d,           // 12b offset for load/store addresses
```

（11）在 el2_exu.sv 的输入信号中，加入 fp_p

```
input logic          dec_qual_lsu_d,                         // LSU instruction at D.  Use to quiet LSU operands
input el2_mul_pkt_t mul_p,                                  // DEC {valid, operand signs, low, operand bypass}
input el2_div_pkt_t div_p,                                  // DEC {valid, unsigned, rem}
input fp_pkt_t       fp_p,
input logic          dec_div_cancel,                         // Cancel the divide operation

input logic  [31:1]  pred_correct_npc_x,                     // DEC NPC for correctly predicted branch
```

（12）在 src/VeeRwolf/VeeR_EL2CoreComplex/el2_veer.sv 的连接信号中，加入 fp_p

```
724   logic        ifu_i0_pc4;
725
726   el2_mul_pkt_t  mul_p;
727   fp_pkt_t       fp_p;
728   el2_div_pkt_t  div_p;
729   logic          dec_div_cancel;
730
731 `ifndef Pipeline
732   logic [31:0] exu_div_result;
733   logic exu_div_wren;
734 `endif
735
```

（13）对应的，修改 el2_exu_div_ctl.sv 文件，连接信号中添加 fp_p

```
22  (
23   input logic          clk,                    // Top level clock
24   input logic          rst_l,                  // Reset
25   input logic          scan_mode,              // Scan mode
26
27   input el2_div_pkt_t  dp,                     // valid, sign, rem
28   input fp_pkt_t       fp,
29   input logic  [31:0]   dividend,              // Numerator
30   input logic  [31:0]   divisor,               // Denominator
31
32   input logic          cancel,                 // Cancel divide
33
```

## 2．除法器修改

（1）在 exu 文件夹下有一个 fp_adder.sv 文件，将这个 fp_adder 的模块集成进 el2_exu_div_ctl 模块中，所有的信号已经在之前步骤为你连好。

（2）借鉴 project1 随行文件夹中一个叫 el2_exu_div_ctl_2.sv 的文件，借鉴这个文件的思路实现独属的集成方式。

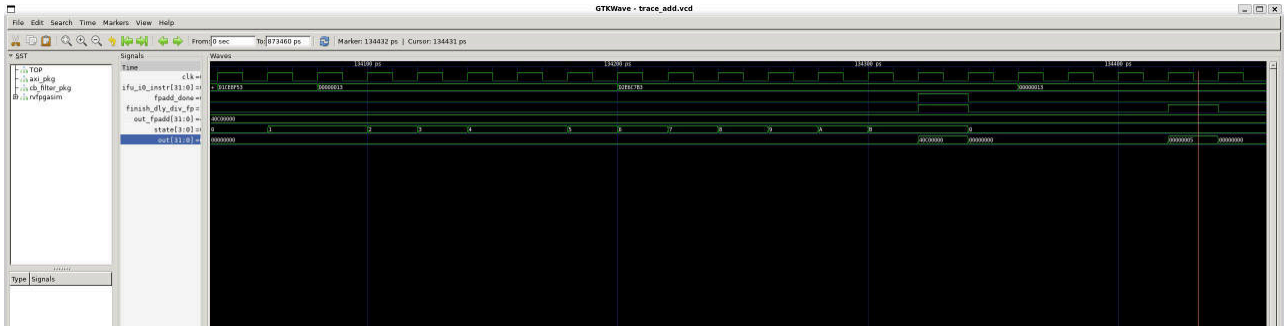
## 3．验证

（1）在 RVfpga_NexysA7-DDR/Labs/Lab18/src 文件夹下，有一个 fp_add.S 的 RISC-V 汇编文件，这个文件便是测试文件。内含重复的 fpadd 指令和 div 指令，前者验证你的浮点加法器集成正确与否，后者测试原来的整数除法器是否仍然正常工作

（2）在你集成好浮点加发器后，在终端通过 cd 进入 Simulators/verilatorSIM_Trace 依次执行：make clean，make，这一步将生成你修改过的 VeeRwolf 工程对应的仿真文件

（3）终端中通过 cd 进入 RVfpga_NexysA7-DDR/Labs/Lab18/commandLine，执行../../../Simulators/verilatorSIM_Trace/Vrvfpgasim +ram_init_file=fp_add.vh +vcd=，过大概 3 秒左右，按下 ctrl+c 终止执行，将看到当前目录下的 trace.vcd 文件已生成

（4）终端中运行 gtkwave trace.vcd，即可打开波形显示器。




Ifu 级取指指令 01CE8F53 代表 fp_add 指令，00000013 代表插入的空指令，02E6C7B3 是整数 DIV 指令。以文件 el2_exu_div_ctl_2.sv 及实现为例,fp_add 在输入浮点数据 0x40800000 和 0x40000000 后生成浮点加结果 out 信号 0x40C00000,结果正确。

DIV 指令对于输入除数 0x00000004 和被除数 0x00000014 后生成除法结果 out 信号 0x00000005。它们都正确生成了结束的 finish 信号

## （三）Bonus：拓展实验

### 1．自己的方式集成浮点 fp_adder 模块

（1）代码编写：整体思路是采用有限状态机来设计

①定义状态机状态

```
127   typedef enum logic {
128       IDLE,       // 空闲状态，等待指令
129       CALC_ADD     // 计算状态，正在执行浮点加法
130   } state_t;
131
132   state_t current_state, next_state;
133
```

②内部信号定义

```
135    logic [31:0] fp_op_a_reg;    // 用于锁存操作数 dividend
136    logic [31:0] fp_op_b_reg;    // 用于锁存操作数 divisor
137    logic [31:0] out_fpadd;      // 浮点加法器的计算结果
138    logic        fpadd_done;     // 浮点加法器的完成信号
139    logic        fp_input_stb;   // 发送给加法器的请求信号
```

③状态机跳转逻辑：

```
142   always_ff @(posedge clk or negedge rst_l) begin
143     if (!rst_l)
144     begin
145       current_state <= IDLE;
146       fp_op_a_reg   <= '0;
147       fp_op_b_reg   <= '0;
148     end
149     else if (cancel)
150     begin
151       current_state <= IDLE; // 如果收到 cancel 信号，强行复位
152     end
153     else
154     begin
155       current_state <= next_state;
156       // 在 IDLE 状态且握手成功时，锁存输入数据
157       if ((current_state == IDLE) && dp.valid && fp.fp_add) begin
158         fp_op_a_reg <= dividend;
159         fp_op_b_reg <= divisor;
160       end
161     end
162   end
```

④有限状态机：分别设定在两种状态下执行的操作

```
165   always_comb begin
166     next_state = current_state; // 保持状态
167     fp_input_stb = 1'b0;        // 默认不请求
168
169     case (current_state)
170       IDLE: begin
171         // 有效指令 (dp.valid)且浮点加法 (fp.fp_add)
172         if (dp.valid && fp.fp_add) begin
173           next_state = CALC_ADD;
174           fp_input_stb = 1'b1; // 拉高 strobe，开始请求
175         end
176       end
177
178       CALC_ADD: begin
179         fp_input_stb = 1'b1; // 在计算期间持续拉高 strobe
180         if (fpadd_done) begin
181           next_state = IDLE; // 计算完成，回到空闲
182         end
183       end
184     endcase
185   end
```

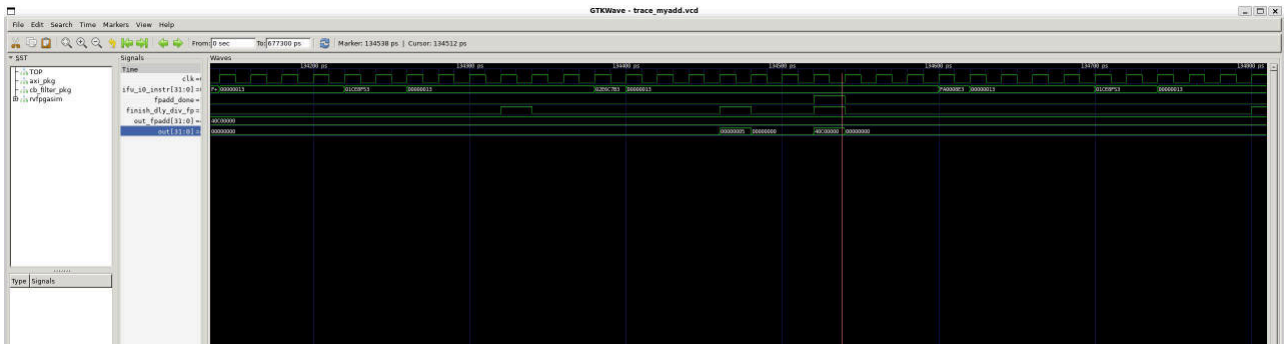
⑤实例化 fp_adder

```
187     // 5. 实例化浮点加法器
188     fp_adder simple_fp_adder (
189         .clk            (clk),
190         .rst            (~rst_l),
191 
192         // 输入数据使用我们锁存好的寄存器值
193         .input_a        (fp_op_a_reg),
194         .input_b        (fp_op_b_reg),
195 
196         // 握手信号：使用状态机生成的时延的 strobe
197         .input_a_stb    (fp_input_stb),
198         .input_b_stb    (fp_input_stb),
199 
200         // 输出接收确认：固定为1
201         .output_z_ack   (1'b1),
202 
203         // 结果输出
204         .output_z       (out_fpadd),
205         .output_z_stb   (fpadd_done),
206 
207         // 输入确认信号
208         .input_a_ack    (),
209         .input_b_ack    ()
210     );
```

⑥选择输出：

```
214     always_comb begin
215         if (fpadd_done) begin
216             // 浮点加法完成
217             finish_dly_div_fp = 1'b1;
218             out[31:0]         = out_fpadd[31:0];
219         end else begin
220             // 整数除法完成（或都没完成，此时 finish_dly 为 0）
221             finish_dly_div_fp = finish_dly;
222             out[31:0]         = out_raw[31:0] & {32{finish_dly}}; // 只有完成时才输出有效数据
223         end
224     end
```

（2）结果分析：




对照发现，fp_add 在输入浮点数据 0x40800000 和 0x40000000 后生成浮点加结果 out 信

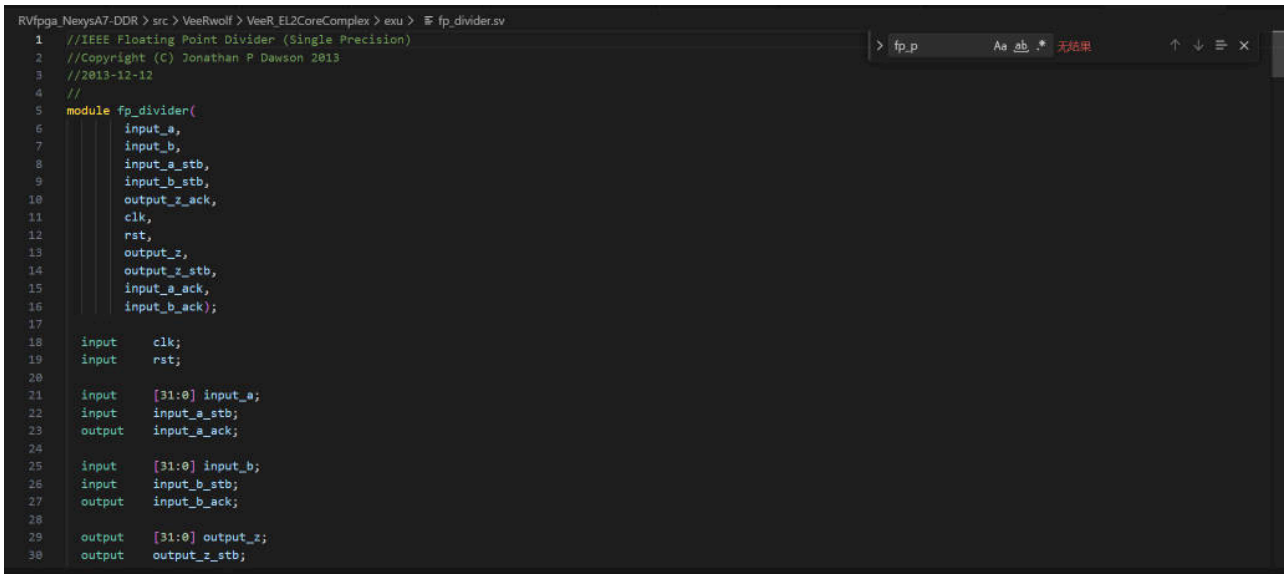
号 0x40C00000,结果正确。

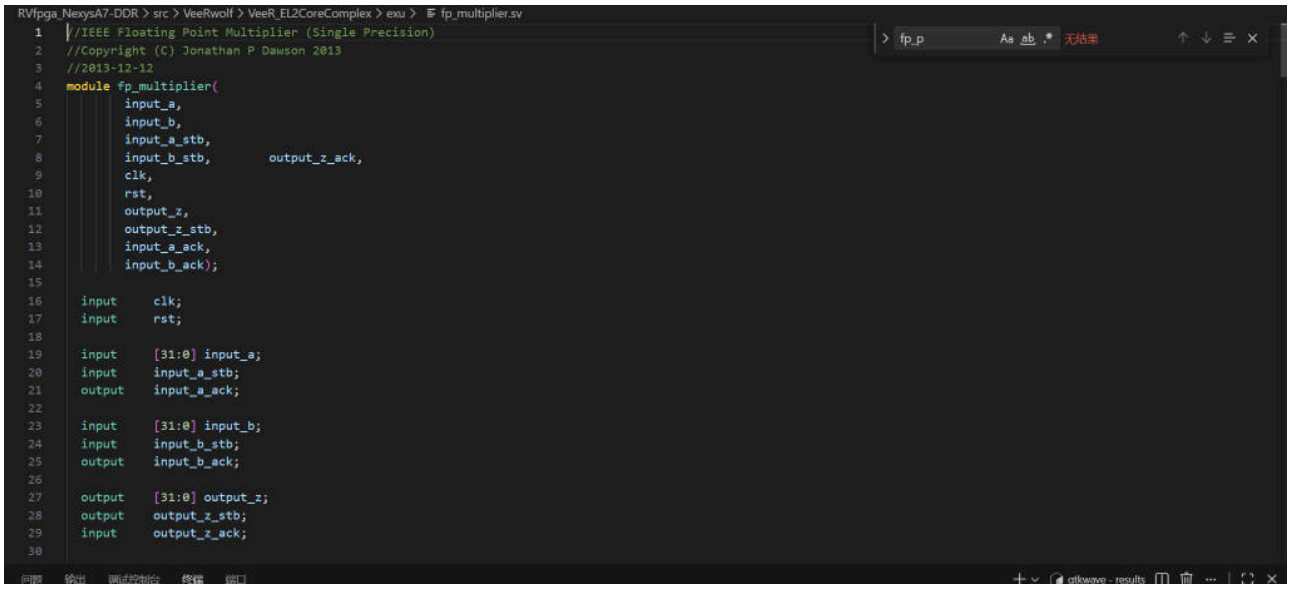
DIV 指令对于输入除数 0x00000004 和被除数 0x00000014 后生成除法结果 out 信号 0x00000005。它们都正确生成了结束的 finish 信号

## 2．增加 fp_mul 和 fp_div 的指令

（1）代码编写：整体思路扔采用有限状态机来设计，因为状态机的可拓展性比较强，而原始的集成代码的锁存和延时比较固定，难以套用到 mul 和 div 上

①在 exu 文件中添加 fp_mul 和 fp_div，注意还需要把 fp_multiplier.sv 和 fp_divider.sv 文件名添加到仿真文件中（写实验报告时实在忘了是哪个文件夹了），否则后续 make 会出错！




②编写集成方式，与我的 add 集成方式类似，只需要修改名称即可

1）Fp_multiplier:

```
// 1. 定义状态机状态
   typedef enum logic {
      IDLE,        // 空闲状态，等待指令
      CALC_MUl     // 计算状态，正在执行浮点加法
   } state_t;

   state_t current_state, next_state;

   // 2. 内部信号定义
   logic [31:0] fp_op_a_reg;    // 用于锁存操作数 dividend
   logic [31:0] fp_op_b_reg;    // 用于锁存操作数 divisor
   logic [31:0] out_fpmul;
   logic        fpmul_done, fpmul_stb;

   // 3. 状态机：状态跳转与数据锁存
   always_ff @(posedge clk or negedge rst_l) begin
      if (!rst_l)
      begin
         current_state <= IDLE;
         fp_op_a_reg    <= '0;
         fp_op_b_reg    <= '0;
      end
      else if (cancel)
      begin
         current_state <= IDLE; // 如果收到 cancel 信号，强行复位
      end
      else
      begin
         current_state <= next_state;
```

```
        // 在 IDLE 状态且握手成功时，锁存输入数据
        if ((current_state == IDLE) && dp.valid && fp.fp_mul) begin
            fp_op_a_reg <= dividend;
            fp_op_b_reg <= divisor;
        end
    end
end

// 4. 状态机：组合逻辑
always_comb begin
    next_state = current_state; // 保持状态
    fp_input_stb = 1'b0;          // 默认不请求

    case (current_state)
        IDLE: begin
            if (dp.valid && fp.fp_mul) begin
                next_state = CALC_MUL;
                fp_input_stb = 1'b1; // 拉高 strobe，开始请求
            end
        end

        CALC_MUL: begin
            fp_input_stb = 1'b1; // 在计算期间持续拉高 strobe
            if (fpmul_done) begin
                next_state = IDLE; // 计算完成，回到空闲
            end
        end
    endcase
end
```

```
// 5. 实例化浮点乘法器
fp_multiplier simple_fp_multiplier (
    .clk              (clk),
    .rst              (~rst_l),

    // 输入数据使用我们锁存好的寄存器值
    .input_a          (fp_op_a_reg),
    .input_b          (fp_op_b_reg),

    // 握手信号：使用状态机生成的时延的 strobe
    .input_a_stb      (fp_input_stb),
    .input_b_stb      (fp_input_stb),

    // 输出接收确认：固定为 1
    .output_z_ack     (1'b1),

    // 结果输出
    .output_z         (out_fpmul),
    .output_z_stb     (fpmul_done),

    // 输入确认信号
    .input_a_ack      (),
    .input_b_ack      ()
);

// 输出仲裁逻辑

always_comb begin
    if (fpmul_done) begin
```

```
        // 浮点除法完成
        finish_dly_div_fp = 1'b1;
        out[31:0]           = out_fpmul[31:0];
    end else begin
        // 整数除法完成
        finish_dly_div_fp = finish_dly;
        out[31:0]           = out_raw[31:0] & {32{finish_dly}}; // 只有完成时才输出有效数据
    end
end
```

2）Fp_divider:

```
// 1. 定义状态机状态
typedef enum logic {
    IDLE,       // 空闲状态，等待指令
    CALC_DIV    // 计算状态，正在执行浮点加法
} state_t;

state_t current_state, next_state;

// 2. 内部信号定义
logic [31:0] fp_op_a_reg;   // 用于锁存操作数 dividend
logic [31:0] fp_op_b_reg;   // 用于锁存操作数 divisor
logic [31:0] out_fpdiv;
logic        fpdiv_done, fp_input_stb;

// 3. 状态机：状态跳转与数据锁存
always_ff @(posedge clk or negedge rst_l) begin
    if (!rst_l)
    begin
        current_state <= IDLE;
        fp_op_a_reg   <= '0;
```

```
        fp_op_b_reg     <= '0;
    end
    else if (cancel)
    begin
        current_state <= IDLE; //  如果收到  cancel  信号，强行复位
    end
    else
    begin
        current_state <= next_state;
        //  在  IDLE  状态且握手成功时，锁存输入数据
        if ((current_state == IDLE) && dp.valid && fp.fp_div) begin
            fp_op_a_reg <= dividend;
            fp_op_b_reg <= divisor;
        end
    end
end

// 4. 状态机：组合逻辑
always_comb begin
    next_state = current_state; //  保持状态
    fp_input_stb = 1'b0;           //  默认不请求

    case (current_state)
        IDLE: begin
            if (dp.valid && fp.fp_div) begin
                next_state = CALC_DIV;
                fp_input_stb = 1'b1; //  拉高  strobe，开始请求
            end
        end
```

```
        CALC_DIV: begin
            fp_input_stb = 1'b1; //  在计算期间持续拉高  strobe
            if (fpdiv_done) begin
                next_state = IDLE; //  计算完成，回到空闲
            end
        end
    endcase
end

// 5.  实例化除点加法器
fp_divider simple_fp_divider (
    .clk              (clk),
    .rst              (~rst_l),

    //  输入数据使用我们锁存好的寄存器值
    .input_a          (fp_op_a_reg),
    .input_b          (fp_op_b_reg),

    //  握手信号：使用状态机生成的时延的  strobe
    .input_a_stb      (fp_input_stb),
    .input_b_stb      (fp_input_stb),

    //  输出接收确认：固定为 1
    .output_z_ack     (1'b1),

    //  结果输出
    .output_z         (out_fpdiv),
    .output_z_stb     (fpdiv_done),
```

```
    // 输入确认信号
    .input_a_ack    (),
    .input_b_ack    ()
);

// 输出仲裁逻辑

always_comb begin
    if (fpdiv_done) begin
        // 浮点除法完成
        finish_dly_div_fp = 1'b1;
        out[31:0]           = out_fpdiv[31:0];
    end else begin
        // 整数除法完成
        finish_dly_div_fp = finish_dly;
        out[31:0]           = out_raw[31:0] & {32{finish_dly}}; // 只有完成时才输出有效数据
    end
end
```

③编写测试文件：

1）Fp_div：

```
#define INSERT_NOPS_1       nop;
#define INSERT_NOPS_2       nop; INSERT_NOPS_1
#define INSERT_NOPS_3       nop; INSERT_NOPS_2
#define INSERT_NOPS_4       nop; INSERT_NOPS_3
#define INSERT_NOPS_5       nop; INSERT_NOPS_4
#define INSERT_NOPS_6       nop; INSERT_NOPS_5
#define INSERT_NOPS_7       nop; INSERT_NOPS_6
#define INSERT_NOPS_8       nop; INSERT_NOPS_7
#define INSERT_NOPS_9       nop; INSERT_NOPS_8
#define INSERT_NOPS_10      nop; INSERT_NOPS_9
```

```
.globl main
main:
    # 准备浮点操作数
    li t3, 0x40800000
    li t4, 0x40000000

    # 准备整数除法操作数
    li a3, 20
    li a4, 4

REPEAT:
    # 浮点除法 (FDIV.S)
    # 操作: f30 = f29(2.0) / f28(4.0) = 0.5 (0x3F000000)
    # 机器码: 0x19ce8f53
    # ----------------------------------------------------------
    INSERT_NOPS_4
    .word 0x19ce8f53

    # ----------------------------------------------------------
    # 整数除法 (DIV)
    # 操作: a5 = 20 / 4 = 5
    INSERT_NOPS_4
    div a5, a3, a4

    # 循环
    INSERT_NOPS_10
    beq zero, zero, REPEAT
```

2）Fp_mul：

```
#define INSERT_NOPS_1           nop;
```

```
#define INSERT_NOPS_2          nop; INSERT_NOPS_1
#define INSERT_NOPS_3          nop; INSERT_NOPS_2
#define INSERT_NOPS_4          nop; INSERT_NOPS_3
#define INSERT_NOPS_5          nop; INSERT_NOPS_4
#define INSERT_NOPS_6          nop; INSERT_NOPS_5
#define INSERT_NOPS_7          nop; INSERT_NOPS_6
#define INSERT_NOPS_8          nop; INSERT_NOPS_7
#define INSERT_NOPS_9          nop; INSERT_NOPS_8
#define INSERT_NOPS_10         nop; INSERT_NOPS_9

.globl main
main:
    # 准备浮点操作数
    li t3, 0x40800000
    li t4, 0x40000000

    # 准备整数除法操作数
    li a3, 20
    li a4, 4

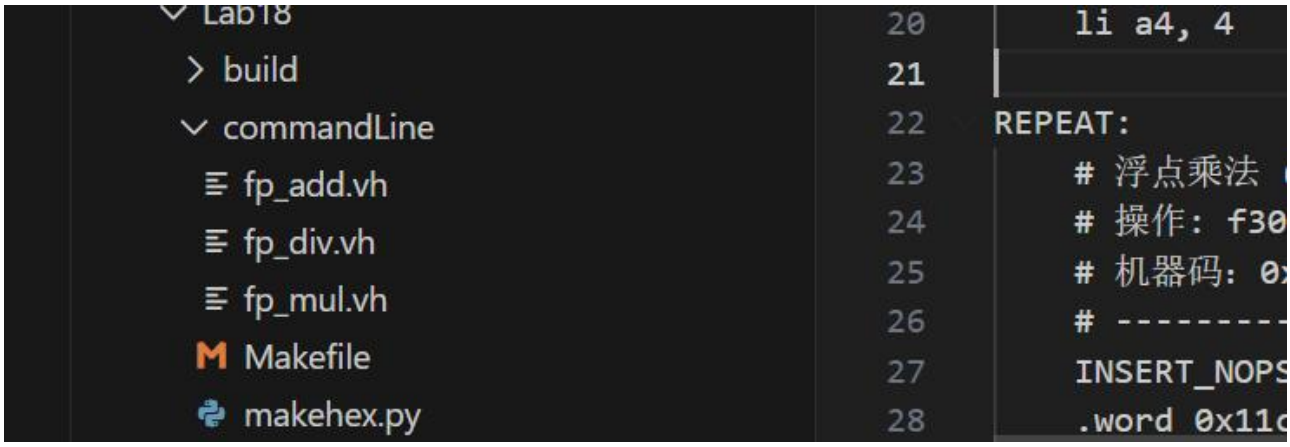
REPEAT:
    # 浮点乘法 (FMUL.S)
    # 操作: f30 = f29(2.0) * f28(4.0) = 8.0 (0x41000000)
    # 机器码：0x11ce8f53
    # ------------------------------------------------------
    INSERT_NOPS_4
    .word 0x11ce8f53

    # 整数除法 (DIV)
```

```
# 操作: a5 = 20 / 4 = 5
INSERT_NOPS_4
div a5, a3, a4

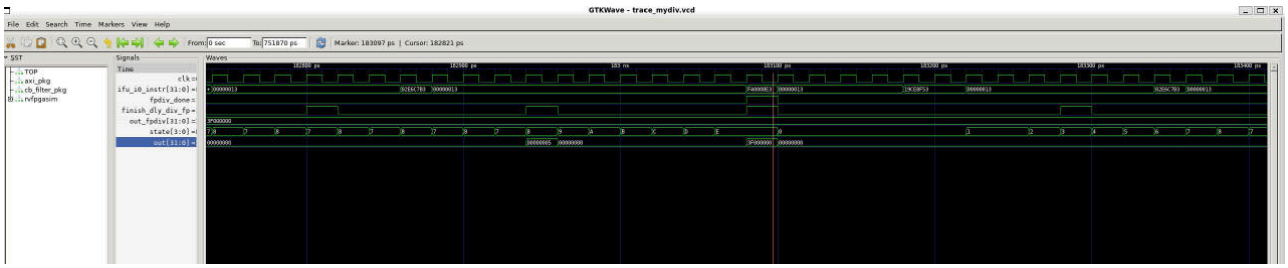
# 循环
INSERT_NOPS_10
beq zero, zero, REPEAT
```

④根据测试文件，生成对应的.vh 文件，即 bonus3 的内容




（2）结果分析：

①Fp_div：

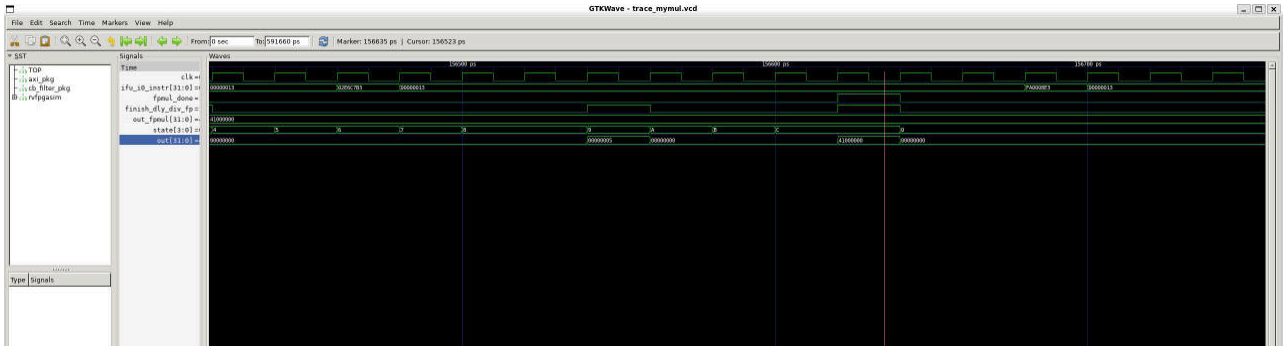



02E6C7B3 是整数 DIV 指令，19ce8f53 是浮点除指令，fp_div 在输入浮点数据 0x40800000 和 0x40000000 后生成浮点除结果 out 信号 0x3F000000（2÷4=0.5）,结果正确。

DIV 指令对于输入除数 0x00000004 和被除数 0x00000014 后生成除法结果 out 信号 0x00000005。它们都正确生成了结束的 finish 信号

其中 state7 和 8 一直循环，表明正在持续做除法，直到设定的精确度

②Fp_mul：



02E6C7B3 是整数 DIV 指令，11ce8f53 是浮点乘指令，fp_mul 在输入浮点数据 0x40800000 和 0x40000000 后生成浮点除结果 out 信号 0x41000000（2*4=8.0）,结果正确。

DIV 指令对于输入除数 0x00000004 和被除数 0x00000014 后生成除法结果 out 信号 0x00000005。它们都正确生成了结束的 finish 信号

## 3．新的测试文件的编写和测试

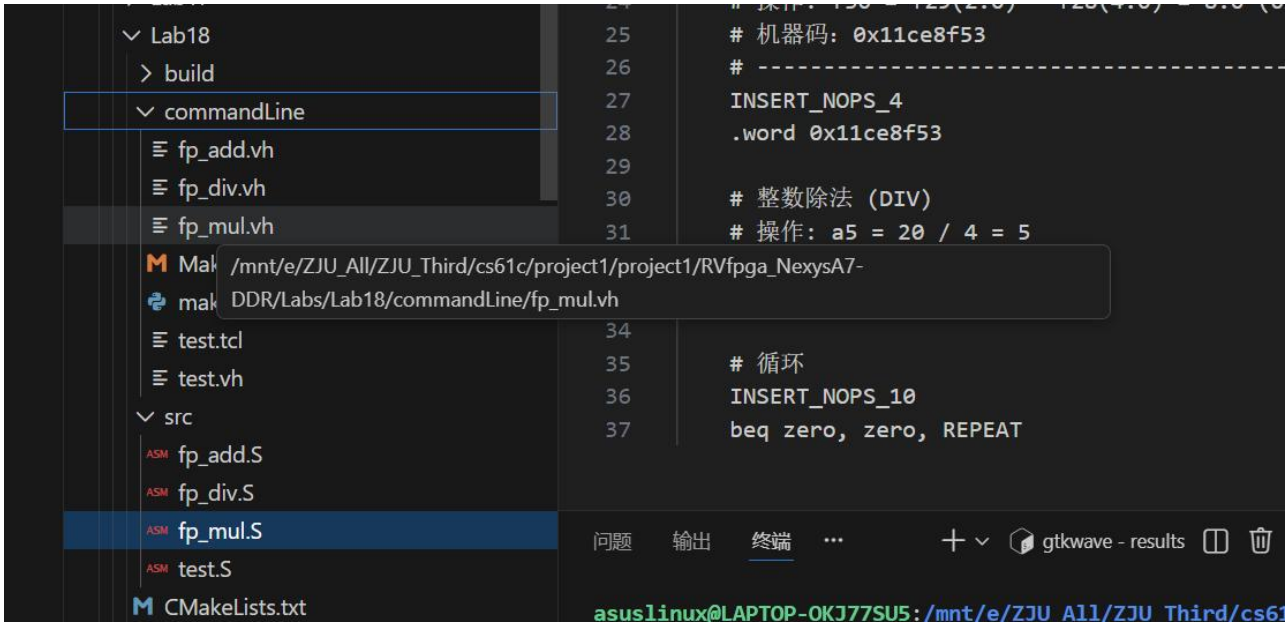
（1）终端依次运行：cd ~/catapult-sdk_examples/1.9.1，source ./set_sdk_path.sh，catapult-studio

（2）修改 CmakeLists.txt 文件，将下图中的 fp_add 修改为 fp_mul，fp_div，修改之后，点击状态栏里的 build，系统会自动启动面向嵌入式的 riscv 编译器完成编译

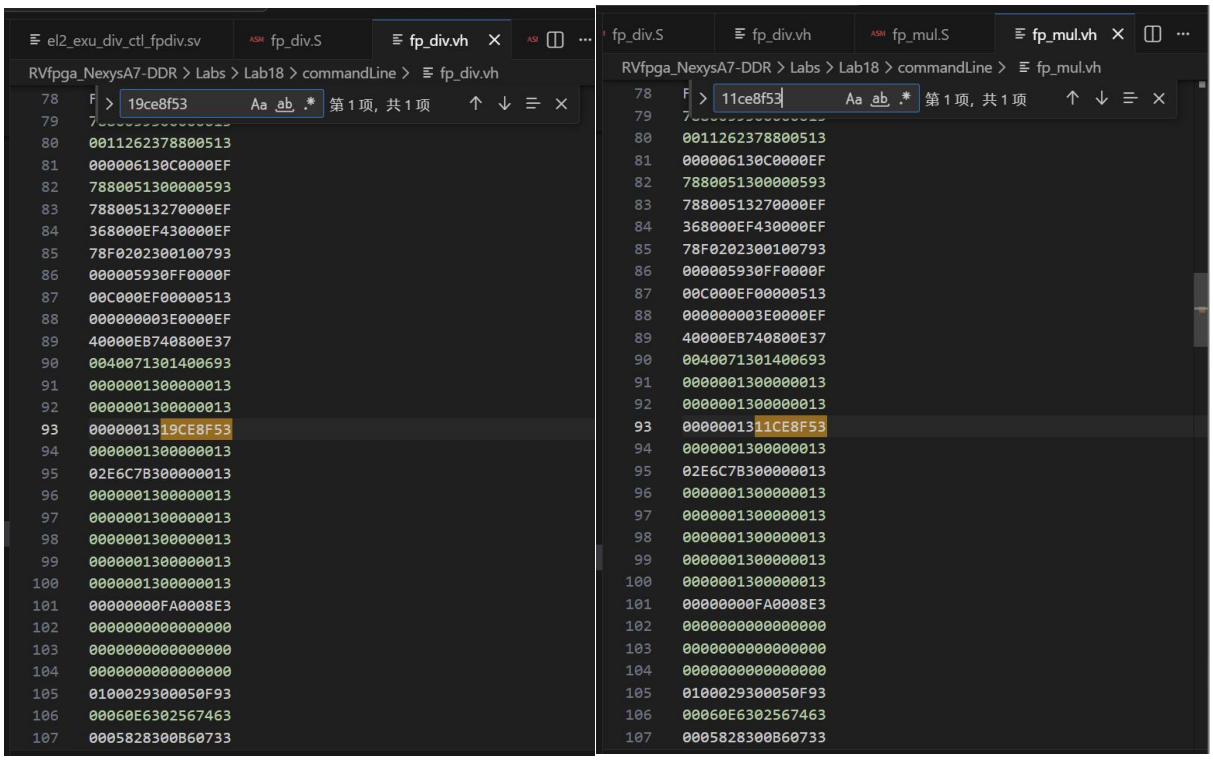
（3）cd 进入 commandLine 文件夹，终端输入:make test.vh 之后你便可以通过在 commandLine 目录下运行 ../../../Simulators/verilatorSIM_Trace/Vrvfpgasim +ram_init_file=test.vh +vcd=1

（4）结果分析：

①生成.vh 文件



②查看.vh 文件中是否还有新增的 mul 和 div 指令，查询后的确存在，因此符合




# 三、总结与感悟

本次实验基于 VeeRwolf 开源 SoC 项目，我成功完成了 RISC-V 浮点指令（fadd、fmul、fdiv）的硬件扩展与验证。在环境搭建阶段，我配置了 WSL2 及 Verilator 仿真环境。在核心设计阶段，我深入剖析了 EL2 处理器的译码与执行逻辑，修改了 el2_dec_decode_ctl.sv 以支持新指令的译码，并在执行级 el2_exu_div_ctl.sv 中，创新性地采用有限状态机（FSM）思路完成了浮点加减乘除模块的集成，解决了原始逻辑难以复用的问题。

实验过程中，Debug 环节让我印象最深。初期因未将 fp_divider.sv 加入编译路径导致 Verilator 报错，让我意识到工程构建细节的重要性。随后在仿真中，遇到指令被取出但硬件不动作的问题，通过 GTKWave 观察波形，我发现是译码级握手信号 dp.valid 未正确拉高导致状态机卡死。修复该逻辑漏洞后，加法与乘除法均输出了正确结果。这次实验不仅让我掌握了 RISC-V 指令扩展的完整流程，更锻炼了通过波形分析定位底层硬件逻辑 Bug 的能力，对计算机体系结构有了更深刻的理解。